PROJECT SYNOPSIS

STUDYFORGE AI

An Intelligent Study Planning and Learning Support Platform

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

Submitted By

[Student Name] | Roll No.: [Roll Number]

Under the Guidance of

[Faculty Guide Name]

[Designation]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AGC AMRITSAR

Academic Session: 2026–27

INDEX

1. INTRODUCTION 3

2. PROBLEM STATEMENT 3

3. OBJECTIVES 4

4. PROPOSED SYSTEM AND METHODOLOGY 4

5. SYSTEM DESIGN 6

6. TECHNOLOGIES AND LIBRARIES USED 7

7. HARDWARE AND SOFTWARE REQUIREMENTS 8

8. EXPECTED OUTCOME AND DELIVERABLES 9

9. WORK PLAN AND TIMELINE 10

10. REFERENCES 10

1. INTRODUCTION

1.1 Overview

StudyForge AI is a full-stack, AI-assisted study workspace that converts uploaded PDF learning material into cited answers, quiz practice, study plans, and visible progress momentum. The system combines a Next.js client application with a FastAPI backend, PostgreSQL-compatible persistence, document extraction, secure authentication, and containerised deployment. It gives students one coherent workspace for their notes, questions, quizzes, flashcards, and planned study tasks.

1.2 Background and Motivation

Students frequently divide their attention between multiple subjects, deadlines, examinations, and learning resources. Generic task applications help record reminders but do not understand academic context, while large online learning platforms often focus on content delivery rather than a student's personal workflow. This project addresses the need for a lightweight, student-centred workspace that brings study planning, progress monitoring, and intelligent guidance together in one interface.

The motivation for StudyForge AI is to reduce the friction between uploading course notes and beginning an effective, focused study session. By maintaining structured records of documents, page-wise text chunks, quiz performance, tasks, XP, and learning preferences, the platform provides contextual and measurable support while preserving source traceability.

1.3 Existing Solutions and Differentiation

Notion and Trello support flexible note-taking and task boards, but they do not provide an academic-domain data model or built-in AI study workflow.

Google Classroom and Moodle distribute course content and assignments, but are mainly institution-led rather than personalised study planners.

General-purpose AI chatbots can answer questions, but often lack source-grounding, PDF citations, quiz-history storage, and structured study-task progress.

StudyForge AI combines a structured learning database with a modern API-first architecture. Its key distinction is grounded learning support: it extracts uploaded PDFs page by page, retrieves the most relevant chunks, and presents answers with source citations. It also combines this with quiz submission, XP/streak tracking, task completion, and an elegant responsive user interface.

2. PROBLEM STATEMENT

Students need a unified and reliable system to transform scattered PDF notes into useful learning activities, manage subject-wise tasks, monitor progress, and obtain timely, source-grounded answers. Existing tools are often fragmented: one tool stores files, another tracks tasks, and another provides AI answers without evidence. This fragmentation makes it difficult to sustain a consistent study routine, verify answers, or identify unfinished priorities.

The problem is important because poor planning and lack of visibility into progress can cause missed deadlines, ineffective revision, and unnecessary academic stress. StudyForge AI aims to

solve this problem by providing a secure, extensible backend that centralises student information and supports an intelligent learning workflow.

3. OBJECTIVES

3.1 Primary Objectives

Develop a RESTful backend for StudyForge AI using FastAPI with documented, health-checkable API endpoints.

Implement JWT-based signup/login and protected routes with securely hashed passwords.

Accept PDF uploads, extract page-wise text using PyMuPDF, and store searchable source chunks with document metadata.

Generate grounded answers, quizzes, flashcards, study-plan tasks, and measurable XP/streak progress from a student's learning material.

3.2 Secondary Objectives

Deliver a polished, responsive Next.js application for library, workspace, quiz, flashcard, dashboard, and planning workflows.

Support a production LLM provider through an environment-controlled OpenAI-compatible API integration, while retaining a safe deterministic mock-AI mode for demonstrations.

Deploy repeatably with Docker Compose, PostgreSQL, and pgvector; retain a SQLite option for lightweight local development.

4. PROPOSED SYSTEM AND METHODOLOGY

4.1 Proposed Approach

The proposed system follows an API-first, layered architecture. A student interacts with a responsive Next.js interface. Requests are sent to the FastAPI application, where route handlers validate input, enforce JWT authentication, and invoke the appropriate business logic. SQLAlchemy acts as the object-relational mapper between Python models and PostgreSQL/SQLite. Docker Compose provisions the database, backend, and frontend as separate services for repeatable execution.

When a PDF is uploaded, the backend uses PyMuPDF to extract text one page at a time and divides it into manageable chunks. A retrieval routine scores chunks against the user's question and supplies the most relevant text as cited evidence. The current demonstration configuration uses deterministic mock quiz content and extractive cited chat; the service layer is intentionally prepared for a production LLM provider for schema-validated quizzes, flashcards, study plans, and explanations.

4.2 Workflow

Step	System Activity	Outcome
1	User signs up or logs in through protected client pages.	JWT-authenticated learner session.
2	Student uploads a subject PDF to the personal library.	Document metadata and page-wise chunks are stored.
3	Student asks a question or opens the workspace.	Relevant chunks are retrieved and cited in the answer.
4	Student generates and submits a quiz or reviews flashcards.	Score, XP, and learning feedback are recorded.
5	Student creates and completes study-plan tasks.	Updated progress, task state, and dashboard metrics.
6	Dashboard presents pending work and momentum.	Clear, personalised next actions for the learner.

4.3 Methodology

Requirement analysis: identify student workflows, data fields, privacy needs, and the minimum viable feature set.

Backend modelling: define SQLAlchemy entities for users, documents, chunks, quiz attempts, and study tasks, with database session management.

API development: implement versioned FastAPI endpoints with Pydantic schemas, dependency injection, JWT security, and automatic OpenAPI documentation.

Grounded intelligence layer: retrieve uploaded source chunks before answering; use an environment-controlled LLM provider only for approved generative workflows.

Testing and deployment: validate APIs using pytest and FastAPI TestClient; package the service with Docker and deploy to a managed cloud environment when ready.

The methodology is iterative. Each module is first designed, implemented in isolation, tested through its API contract, and then integrated into the common data model. This reduces coupling and keeps the platform maintainable as functionality grows.

5. SYSTEM DESIGN

5.1 High-Level Architecture

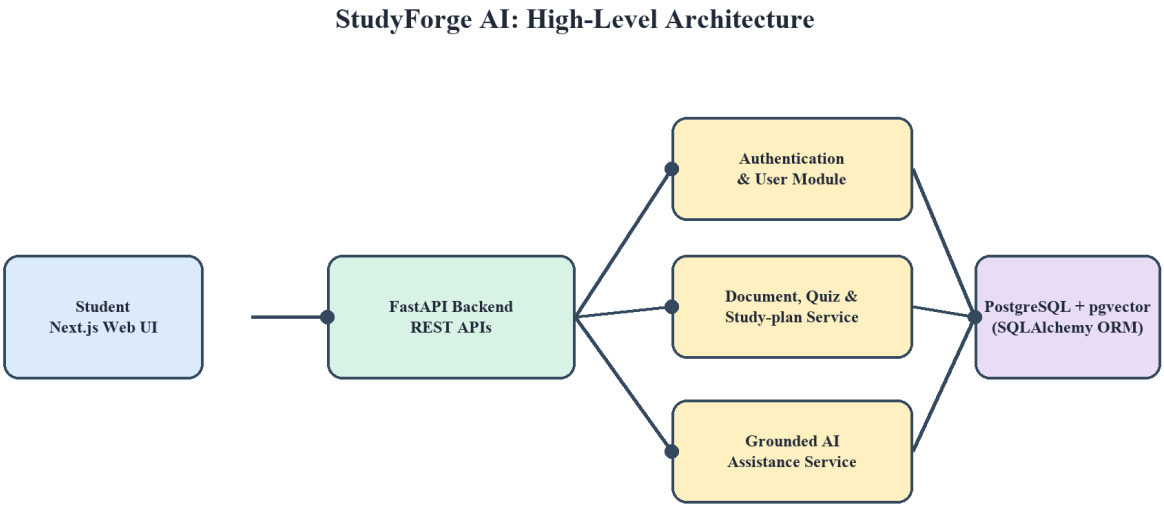


Figure 1: High-level architecture of StudyForge AI.

The client layer is implemented as a responsive Next.js web application. FastAPI is the central service layer and exposes REST endpoints. Authentication, document processing, retrieval, quizzes, planning, and progress modules remain logically separated while sharing the same domain model. SQLAlchemy manages persistence to SQLite for local development and PostgreSQL with pgvector in Docker-based deployments.

5.2 ER Diagram and Database Schema

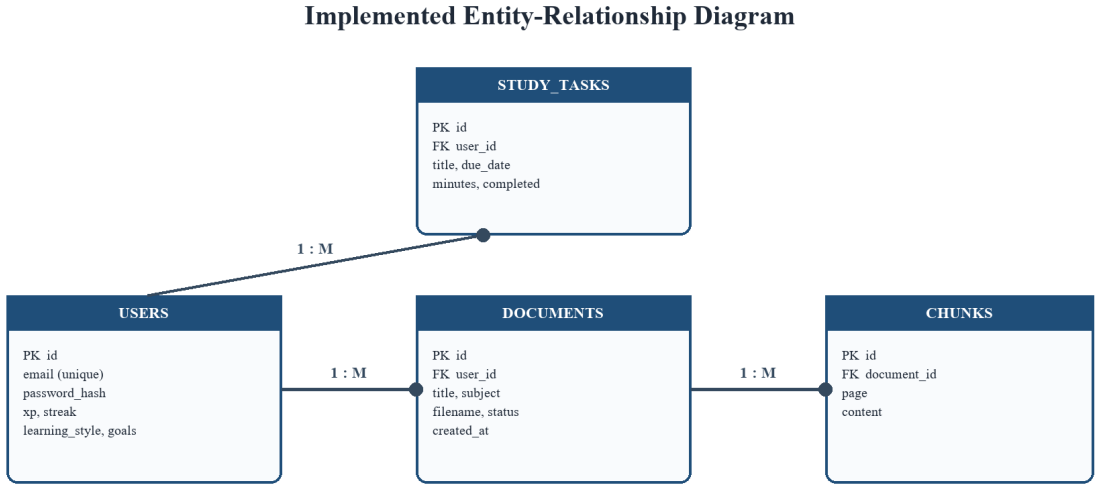


Figure 2: Proposed entity-relationship diagram.

The implemented database includes USERS, DOCUMENTS, CHUNKS, QUIZ_ATTEMPTS, and STUDY_TASKS. The user record stores secure identity and learning-preference information, documents belong to users, and a document contains many page-wise chunks. Quiz attempts store score and total marks, while study tasks store due date, estimated duration, and completion state.

Entity	Key Fields	Purpose
USERS	id, email, password_hash, xp, streak, learning_style	Stores secure learner identity and preferences.
DOCUMENTS	id, user_id, title, subject, filename, status	Stores uploaded PDF metadata.
CHUNKS	id, document_id, page, content	Stores extracted source text for cited retrieval.
QUIZ_ATTEMPTS	id, user_id, document_id, score, total	Stores assessment performance.
STUDY_TASKS	id, user_id, title, due_date, minutes, completed	Represents actionable study work.

5.3 Data Flow Description

At Level 0, the Student supplies account details, PDF files, questions, quiz responses, and task data to StudyForge AI, and receives cited answers, assessment feedback, plans, and progress metrics. At Level 1, the backend routes account data to Authentication, files to Document Processing, questions to Grounded Retrieval, and learning actions to the Quiz/Planning modules. Each module reads or writes controlled records through the data-access layer. The resulting response is returned to the Next.js client in JSON format.

6. TECHNOLOGIES AND LIBRARIES USED

StudyForge AI is intentionally built with current, industry-relevant technologies. The present codebase already uses the following core stack, while the proposed stack supports future delivery of the full intelligent platform.

Technology / Library	Role in StudyForge AI	Status
Python 3	Primary backend programming language; provides readable, maintainable application logic.	Used
FastAPI	High-performance REST API framework; provides routing, validation integration, automatic Swagger/OpenAPI documentation, and async-ready architecture.	Used
Uvicorn	ASGI server used to run the FastAPI application during development and deployment.	Used
SQLAlchemy	ORM for declarative data models, database engine configuration, sessions, and portable queries.	Used

Technology / Library	Role in StudyForge AI	Status
PostgreSQL + pgvector	Containerised relational database and vector-extension-ready persistence; SQLite remains supported for local development.	Used
Pydantic Settings	Configuration and API schema support for validated environment-driven application settings.	Used
JWT + python-jose + passlib/bcrypt	Token-based authentication and secure password hashing for protected account workflows.	Used
PyMuPDF (fitz)	Page-wise PDF text extraction and chunking of learner-uploaded source material.	Used
OpenAI SDK / compatible LLM API	Production-ready integration point for generative study plans, quizzes, flashcards, and explanations; mock-AI mode is enabled for safe demos.	Used / configurable
Next.js 15 + React 19	App Router-based responsive frontend for landing, library, workspace, quiz, flashcards, dashboard, and planning flows.	Used
TypeScript + Tailwind CSS	Type-safe frontend implementation and responsive dark-first interface styling.	Used
Framer Motion + Lucide React	User-interface motion and accessible iconography.	Used
Docker Compose	Runs frontend, FastAPI backend, PostgreSQL, volumes, and service networking reproducibly.	Used
pytest + httpx	Automated backend/API tests for reliable iterative development.	Used
AWS / GCP / Azure	Future cloud-hosting options for managed database, object storage, monitoring, and scalable deployment.	Planned

FastAPI and SQLAlchemy separate API concerns from storage concerns, while Next.js provides a modern type-safe client experience. PyMuPDF plus the source-chunk retrieval workflow makes the AI experience evidence-aware rather than purely conversational. PostgreSQL/pgvector and Docker Compose create a cloud-native path for growth, and the modular service layer enables future managed deployment on AWS, GCP, or Azure.

7. HARDWARE AND SOFTWARE REQUIREMENTS

7.1 Minimum Hardware Requirements

Processor: Intel Core i3 / AMD equivalent or higher.

RAM: 4 GB minimum; 8 GB recommended for development tools and local testing.

Storage: 2 GB free space for source code, virtual environment, database, and development utilities.

Internet: Required for package installation, Git repositories, cloud deployment, and LLM API access.

7.2 Software Requirements

Operating System: Windows 10/11, Linux, or macOS.

Python 3.10+ with pip and a virtual environment.

VS Code or PyCharm; Git for source control; Postman/Insomnia for API testing.

FastAPI, Uvicorn, SQLAlchemy, and SQLite; Docker Desktop and a modern browser for optional deployment and frontend work.

8. EXPECTED OUTCOME AND DELIVERABLES

The expected outcome is a functioning backend service for StudyForge AI that can run locally, confirm service health, initialise the database, and manage a secure user model. The completed project will demonstrate a complete learner workflow in which a student creates an account, builds study plans, adds tasks, monitors completion, and receives AI-generated academic support where the integration is enabled.

Next.js frontend and FastAPI backend with documented OpenAPI/Swagger endpoints.

JWT authentication, PostgreSQL/SQLite persistence, and SQLAlchemy models for users, PDFs, chunks, attempts, and tasks.

PDF library, cited workspace chat, quiz generation/submission, flashcards, study-plan tasks, and XP/streak progress modules.

Database schema/ER diagram, API test collection, project report, and demonstration video or live demo.

Optional Docker configuration and cloud deployment guide for reproducible execution.

9. WORK PLAN AND TIMELINE

Phase	Weeks	Activities and Milestone
I. Analysis	1–2	Collect requirements, review similar systems, finalise use cases and scope.
II. Design	3–4	Prepare database schema, API contracts, architecture, and wireframes.
III. Core Backend	5–7	Implement FastAPI, SQLAlchemy, JWT authentication, document uploads, and tests.
IV. Learning Modules	8–10	Build PDF chunking, cited retrieval, quizzes, tasks, progress, and dashboard APIs.
V. AI Integration	11–12	Add guarded LLM workflows for generated quizzes, flashcards, planning, and explanations.
VI. UI and Integration	13–14	Build and connect the Next.js client; conduct end-to-end integration testing.
VII. Deployment & Review	15–16	Dockerise, perform final testing, prepare report, synopsis, and demonstration.

Key milestones are: approved design by Week 4, operational authenticated document backend by Week 7, complete cited learning workflow by Week 10, AI-assisted prototype by Week 12, and final containerised demonstration by Week 16.

10. REFERENCES

FastAPI Documentation. https://fastapi.tiangolo.com/ (accessed July 2026).

SQLAlchemy Documentation. https://docs.sqlalchemy.org/ (accessed July 2026).

SQLite Documentation. https://www.sqlite.org/docs.html (accessed July 2026).

Uvicorn Documentation. https://www.uvicorn.org/ (accessed July 2026).

OpenAI API Documentation. https://platform.openai.com/docs/ (for proposed LLM integration; accessed July 2026).

Sommerville, I. Software Engineering, 10th Edition, Pearson, 2015.